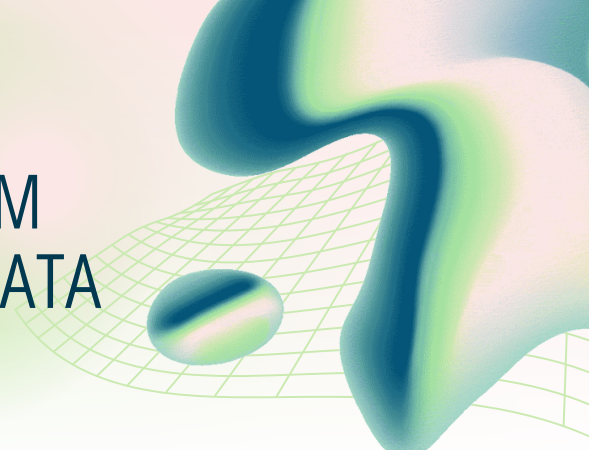


REAL-TIME FACIAL EMOTION RECOGNITION USING A CUSTOM CNN ON CLEANED FER-2013 DATA



From Data-Quality Auditing to Live Webcam Inference

Dhruv — [Rajiv Gandhi Institute of Petroleum Technology] — July 2026

At a Glance

- 26,942 / 7,125 — Cleaned train / test images
- 7 — Emotion classes
- 1,241,895 — Model parameters (GAP architecture)
- 0.60 — Best weighted F1
- 1,853 — Duplicate images caught & removed

Abstract

Facial Emotion Recognition (FER) remains a challenging computer vision task because of the visual overlap between expressions and the severe class imbalance present in benchmark datasets. This report documents the design, training, and real-time deployment of a custom Convolutional Neural Network (CNN) that classifies seven basic emotions — angry, disgust, fear, happy, neutral, sad, and surprise — from the FER-2013 dataset. Beyond model development, this project placed significant weight on data-quality auditing: an MD5 hash-based scan uncovered 1,853 duplicate images, including 531 leaks across the train/test split that would otherwise have inflated evaluation metrics. A four-block CNN using Global Average Pooling(GAP) in place of a Flatten-Dense head cut trainable parameters by nearly 75% while improving generalisation. A two-phase training strategy — an unweighted warm-start followed by damped class-weighting — raised disgust recall from 0.00 to 0.18 without destabilising the other six classes, reaching a final weighted F1 of 0.60 . The trained model was integrated into a live OpenCV pipeline using Haar cascade face detection and confirmed to run correctly end-to-end on webcam input.

1. Introduction

Facial expressions carry a large share of the emotional signal in human communication, and automating their recognition has applications ranging from human-computer interaction and classroom engagement monitoring to accessibility tools and market research. The rise of deep

learning, and convolutional neural networks (CNNs) in particular, has made real-time, camera-based emotion recognition practical on consumer hardware.

This project set out to build such a system end-to-end: from a raw copy of the FER-2013 dataset to a live webcam application that detects a face, classifies its expression into one of seven categories, and displays the result instantly. Two challenges shaped most of the engineering decisions documented here. First, FER-2013 is a famously noisy dataset — mislabeled images, duplicates, and, less commonly checked, leakage between its own train and test splits, all of which can silently inflate reported accuracy if left unaddressed. Second, the dataset's class distribution is heavily skewed toward the disgust class, which made naive training collapse onto majority-class predictions.

Rather than treating these as footnotes, this report documents them as first-class parts of the pipeline: the data audit that found and removed the leakage, the training strategy that stabilised minority-class learning without collapsing the majority classes, and the concrete bugs, and fixes, encountered while getting there. In summary, this project contributes:

- A verified, deduplicated FER-2013 split (26,942 train / 7,125 test) with all cross-split leakage removed.
- A compact 4-block CNN (1,241,895 parameters) using Global Average Pooling, trained with a two-phase class-weighting strategy.
- A documented set of six concrete training and inference pitfalls, together with their fixes.
- A working real-time OpenCV + Haar cascade inference pipeline with a verified train/inference preprocessing contract.

2. Related Work

Several published systems follow a similar FER-2013 to Haar cascade to CNN to OpenCV pipeline. Urgunde et al. report a CNN trained on 28,709 FER-2013 training images that reaches 80.42% training accuracy and 65.34% test accuracy, with a weighted F1 of 0.63 and a disgust recall of 0.46. The comparison below situates this project's results against theirs.

Two details are worth flagging when reading their numbers alongside this project's own. Their paper describes an evaluation set of only 178 images, yet the per-class support values in their own classification-report table sum to 7,178 — matching the size of the standard combined FER-2013 test split rather than 178. This suggests their class-wise metrics, including the 0.46 disgust recall, were computed on the full test split despite the stated description; that is the assumption used for the comparison below.

3. Dataset and Data Quality Audit

FER-2013 is a long-standing benchmark for emotion recognition: 48×48 grayscale face crops labelled into seven categories — angry, disgust, fear, happy, neutral, sad, surprise — typically distributed as roughly 28,700 training images and just over 7,000 test images in its common packaging. It is also widely documented as a noisy dataset: some images are mislabelled, cropped incorrectly, or not faces at all, and, less commonly checked, duplicated across its own splits.

Before any model was trained on the raw download, an **MD5 hash**-based scan was run across every image to catch exact duplicates. It found 1,853 duplicate files, including 531 that appeared in both the train and test folders — a direct leakage problem, since a duplicate sitting in the test set lets the model effectively see that exact image during training and then be evaluated on it again as if it were held out. The same scan surfaced a single source image saved under five different emotion labels across its copies. A separate pass confirmed zero corrupted image files. After removing all duplicates and leaked pairs, the working dataset stood at 26,942 training images and 7,125 test images.

Why this mattered: Cross-split leakage doesn't just add noise — it silently inflates the metric a report is built around. A model can score well on a "held-out" test image it partially memorised during training, and that inflated number would have looked like real progress. Catching this before training started means every accuracy and F1 figure in this report reflects genuinely unseen data.

4. Preprocessing Pipeline

Both training and inference share the same preprocessing contract: convert to grayscale, detect the face region, resize to 48×48, and normalise pixel values to [0, 1]. The normalisation step deserves a specific note, because it is also the source of the one inference bug encountered in this project (Section 8.3): rather than dividing by 255 inside the preprocessing code, a `Rescaling(1./255)` layer was built directly into the model graph as its first layer. That decision means the model always expects raw pixel values (0–255) as input — normalisation happens automatically as part of a forward pass, so any script feeding the model must not normalise a second time upstream. Training-time augmentation (random flips and rotations, applied only to the training split) was used to reduce overfitting risk given the dataset's modest size and the added parameter-efficiency pressure from having removed nearly 2,000 images during cleaning.

5. Model Architecture

5.1 First Iteration (Baseline)

The first architecture used three convolutional blocks feeding into a Flatten layer and a 256-unit Dense layer — a standard, if parameter-heavy, CNN classifier design (~1.27M trainable parameters). Trained without class weights, it reached 43% accuracy, but the confusion matrix told a more specific story than the headline number: both angry and disgust recall sat at 0.00. Ambiguous inputs across several visually similar classes (angry, sad, neutral, fear) were being pulled toward a single default prediction of "sad" rather than being separated by the architecture. That confusion-matrix diagnosis, not the accuracy number, is what motivated the redesign below.

5.2 Final Architecture (Cnn_model_new)

The revised architecture uses four convolutional blocks, each with two stacked convolutions before pooling, and replaces the Flatten → Dense head with Global Average Pooling (GAP) feeding directly into the output layer. GAP collapses each feature map to a single value instead of flattening every activation into a large Dense layer, which cut the parameter count from 3,535,655 to 1,241,859 — a reduction of nearly 75% — while directly addressing the overfitting risk a Flatten → Dense head

introduces on a modestly sized dataset. This became the final architecture used for every subsequent result in this report.

6. Training Methodology

6.1 Two-Phase Training Strategy

Rather than applying class weights from the first epoch, training was split into two phases. Phase 1 (history1) trained the GAP architecture on the cleaned dataset with no class weighting for roughly 50 epochs, reaching 51% accuracy and, notably, moving angry (0.52 recall) and neutral (0.62 recall) off zero, though disgust remained unlearned.

Phase 2 (history2) resumed training from that checkpoint and reintroduced class weights, but at a damped magnitude rather than the raw inverse-frequency "balanced" setting, continuing to a cumulative epoch 61. This phase is what finally moved disgust recall off zero, and the best checkpoint within it, restored from epoch 31 of this phase, became the final selected model.

```
angry      : 1.01
disgust    : 10.65
fear       : 1.01
happy      : 0.54
neutral    : 0.80
sad        : 0.84
surprise   : 1.57
```

#Balanced Class weights

```
angry      : 1.01
disgust    : 3.26
fear       : 1.00
happy      : 0.74
neutral    : 0.89
sad        : 0.91
surprise   : 1.25
```

#Damped Class weight(square root of original balanced class weight)

6.2 Engineering Pitfalls and Fixes

Four concrete issues surfaced during training and inference. Each is recorded here with its fix, since none were obvious from the error messages alone.

1. Class weights on an untrained model. Applying `class_weight='balanced'` from epoch 1 caused the loss to lock at $\ln(7)$ (≈ 1.9459) with near-random accuracy — the large inverse-frequency multiplier on disgust interacted badly with BatchNormalization on an untrained network. Fix: warm-start without weights first, then reintroduce a damped weighting once training has stabilised.

2. Competing callback patience values. Setting EarlyStopping and ReduceLROnPlateau to the same patience made them fire on the same epoch, so training stopped before a learning-rate

reduction had a chance to help. Fix: give ReduceLROnPlateau a tighter patience than EarlyStopping so it can act first.

3. shuffle=False silently dropped a class. Calling `image_dataset_from_directory` with `shuffle=False` before a `validation_split` excluded the entire "surprise" class — the alphabetically last of the seven — from training, since the split point fell before any surprise examples appeared in the unshuffled file order. Fix: shuffle before splitting.

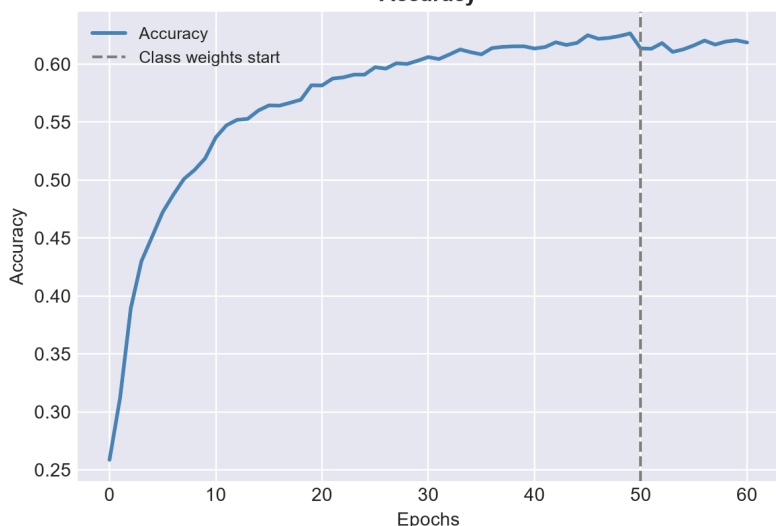
4. cv2.imshow() in Jupyter. OpenCV's display window is unreliable inside Jupyter notebooks. The real-time script was moved to a standalone `.py` file run from VS Code or Anaconda Prompt, where it behaved correctly.

7. Results

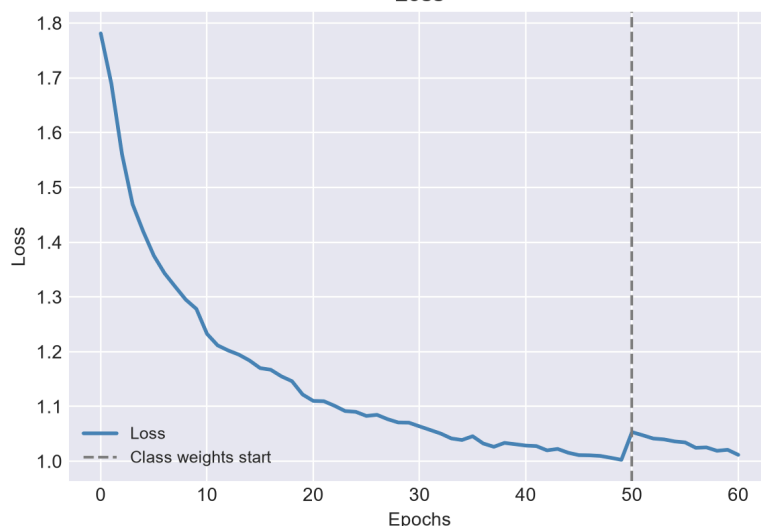
7.1 Training Curves

Training History: Warm-start → Class-weighted Phase

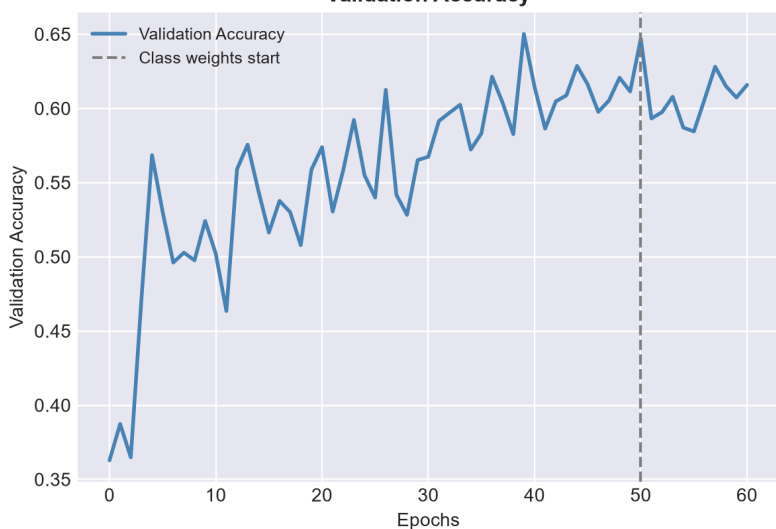
Accuracy



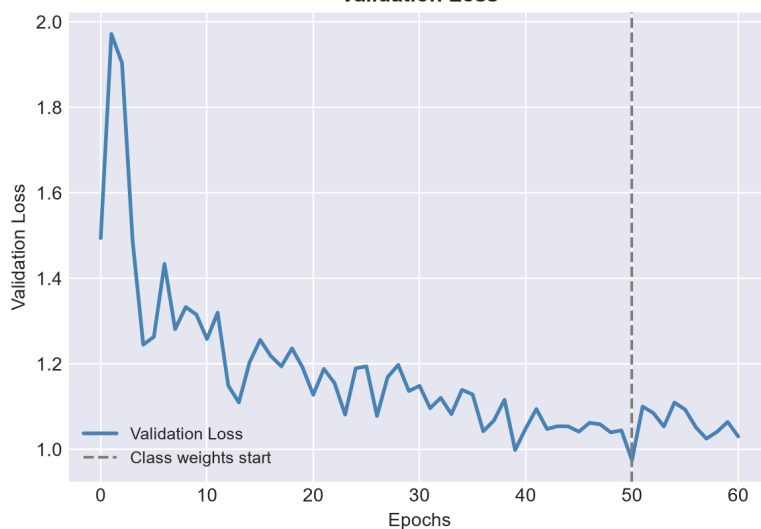
Loss



Validation Accuracy



Validation Loss



Above figures shows accuracy, validation accuracy, loss, and validation loss across both training phases, with the phase-1-to-phase-2 boundary marked. Together they show the practical effect of the warm-start-then-damped-weighting strategy from Section 6.1: the transition point coincides with the disgust recall gain, without the instability that applying weights from epoch 1 produced.

7.2 Classification Reports

Table 2. Model Progression Summary

- Baseline (3-block, Flatten → Dense) — Class weights: None. Accuracy: 43%. Weighted F1: NA. Disgust recall: 0.00.
-
- Final architecture, Phase 1 (history1) — Class weights: None. Accuracy: 51%. Weighted F1: 0.59. Disgust recall: 0.00.
-
- Final architecture, Phase 2 (history2, final) — Class weights: Damped, warm-started. Accuracy: ~58% train / ~55% test. Weighted F1: 0.60 . Disgust recall: 0.18.

Beyond disgust, the damped-weighting phase also lifted **neutral** recall to 0.65, without the other classes regressing — the full per-class breakdown is visible in the pasted classification reports below.

Figure 3: When model trained without class-weight

	precision	recall	f1-score	support
angry	0.50	0.52	0.51	954
disgust	0.00	0.00	0.00	110
fear	0.41	0.31	0.35	1016
happy	0.87	0.79	0.83	1768
neutral	0.52	0.62	0.56	1226
sad	0.46	0.53	0.49	1240
surprise	0.69	0.74	0.71	811
accuracy			0.59	7125
macro avg	0.49	0.50	0.49	7125
weighted avg	0.59	0.59	0.59	7125

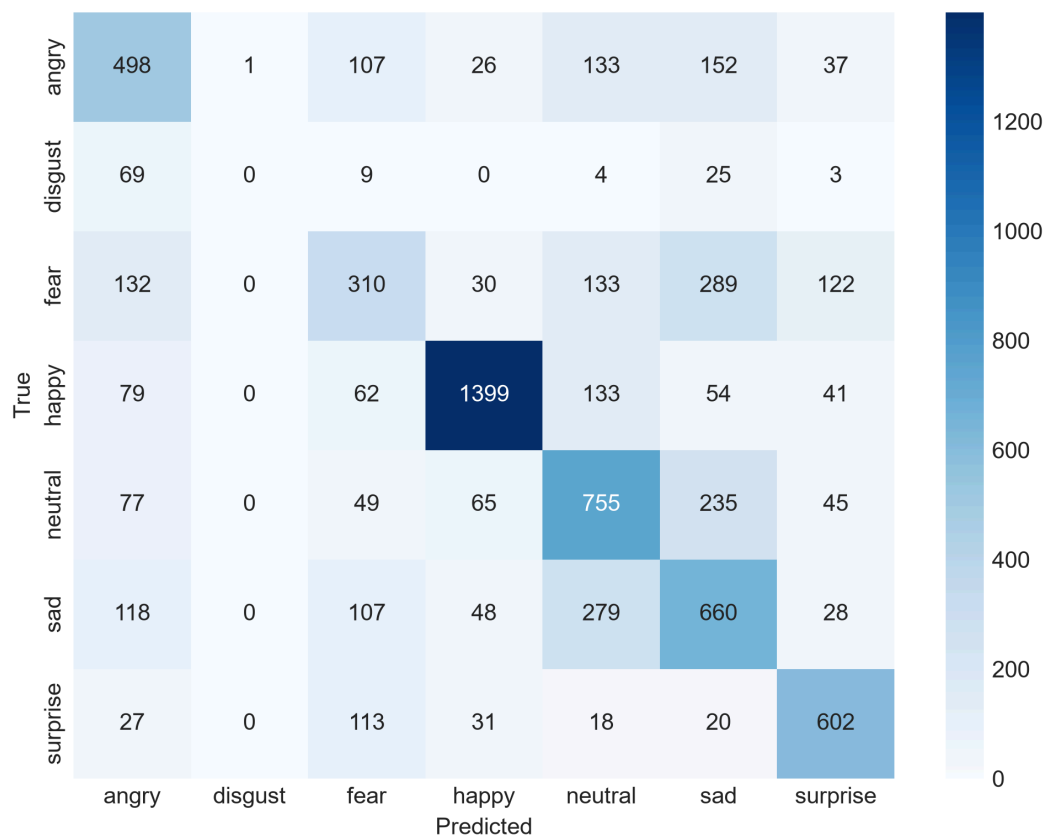
Figure 4: When model trained with Damped-Classweight

	precision	recall	f1-score	support
angry	0.50	0.51	0.50	954
disgust	0.41	0.18	0.25	110
fear	0.42	0.30	0.35	1016
happy	0.90	0.79	0.84	1768
neutral	0.52	0.65	0.57	1226
sad	0.48	0.51	0.50	1240
surprise	0.66	0.77	0.71	811
accuracy			0.60	7125
macro avg	0.55	0.53	0.53	7125
weighted avg	0.60	0.60	0.59	7125

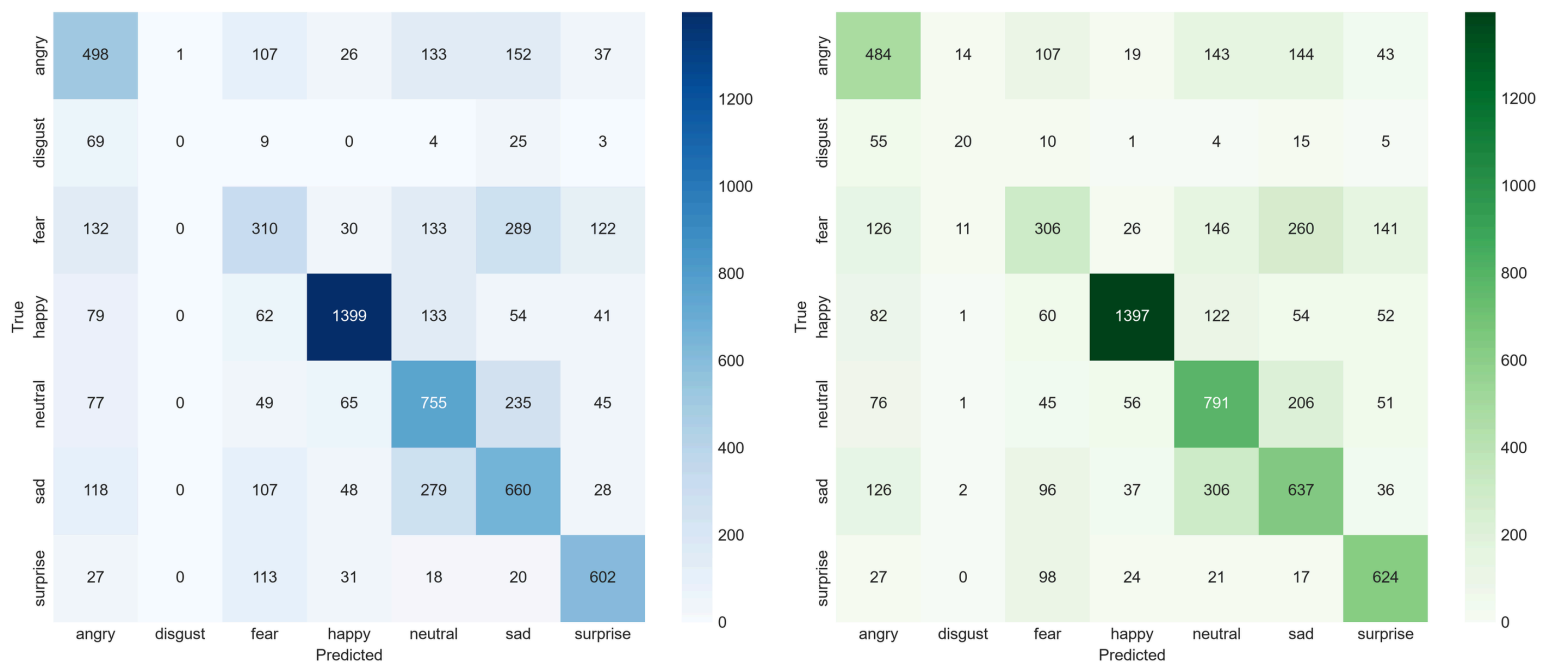
7.3 Confusion Matrices

Figure 5 shows the confusion matrix immediately after Phase 1 (no class weighting), where confusions concentrate around angry, sad, neutral, and fear. Figure 6 places the Phase 1 and Phase 2 confusion matrices side by side to make the effect of class weighting on the disgust column, specifically, visible at a glance.

Figure 5 : confusion matrix for history1, pre class-weighting



Insert Figure 6 : side-by-side confusion matrices for history1 and history2



7.4 The Disgust Ceiling: an Abandoned Oversampling Attempt

A follow-up attempt to raise disgust recall further used oversampling with additional augmentation for the minority class. Across multiple runs, disgust recall collapsed back to 0.00 and training became visibly unstable, while angry, fear, and surprise recall all regressed at the same time relative to the Phase 2 checkpoint. The original checkpoint outperformed every subsequent oversampling iteration on the metrics that mattered, so it was kept as the final model rather than iterated past. This is treated in Section 9 as a deliberate, evidence-based stopping decision rather than an unresolved gap.

8. Real-Time Inference Pipeline

8.1 System Overview

The trained model was wired into a real-time OpenCV application (realtime.py). Each captured frame is flipped for a mirror view, converted to grayscale, and passed to a Haar cascade face detector (scaleFactor = 1.1, minNeighbors = 5, minSize = 48×48). Every detected face region is cropped from the frame, resized to 48×48, reshaped to a (1, 48, 48, 1) batch, and passed to model.predict() — the model's internal Rescaling layer handles normalisation, so the script itself performs no manual scaling. The predicted class and its confidence are drawn back onto the frame as a bounding box and label. This pipeline was confirmed working end-to-end on live webcam input.

Table 3. Walkthrough of script designed for OpenCV

- Normalisation — baked into the model (Rescaling layer)
- Face detection params —scaleFactor 1.1, minNeighbors 5, minSize 48×48.
- Camera handling — DirectShow backend, auto-fallback across indices, --probe scan mode.
- Output display — confidence %, 7-class mini bar chart, per-emotion colour coding.
- Configuration — CLI arguments (--model, --camera).
- Error handling — try/except on model load, graceful exit.
-

9. Discussion

The final model was selected on aggregate metrics — accuracy and weighted F1 — rather than by chasing the single hardest class, and that decision was tested directly rather than assumed: Section 7.4 describes an oversampling attempt that regressed multiple classes at once and was reverted. Choosing the aggregate-best checkpoint over the disgust-focused iteration is the more defensible reading of that evidence.

The final weighted F1 of 0.60 set at the outset of this phase of the project. Given that further disgust-focused iteration was actively making other classes worse, stopping at the best-validated checkpoint was judged the more defensible choice than continuing to chase a number that subsequent attempts showed was not reachable without trading away performance elsewhere.

It is also worth reading this project's ~60% accuracy against a documented reference point: human raters classifying FER-2013-style expressions are themselves estimated to agree with the ground-truth label only around 65% of the time, since several of the seven categories, particularly

fear/surprise and sad/neutral/angry, genuinely overlap in appearance. Against that ceiling, and after removing nearly 2,000 leaked or duplicated images that would otherwise have inflated the test score, a ~59% weighted result across seven classes is a defensible, honestly measured outcome rather than an underperforming one.

10. Limitations

- Disgust remains the weakest class, reflecting FER-2013's severe minority-class share — a documented property of the dataset itself, not a specific architecture failure.
- Haar cascade face detection is sensitive to non-frontal poses, occlusion, and low light;
- The real-time system was verified qualitatively (visual confirmation on live webcam input) rather than benchmarked systematically for latency or frame rate.
- Evaluation is limited to FER-2013.

11. Conclusion

This project delivered a complete facial emotion recognition pipeline — from a raw, unaudited FER-2013 download to a live webcam application — while treating data quality and training stability as first-class engineering problems rather than afterthoughts. Removing 1,853 duplicate images, including 531 cross-split leaks, means every metric reported here reflects genuinely held-out performance. A compact, GAP-based CNN (1,241,895 parameters) and a two-phase warm-start-then-damped-class-weighting strategy moved disgust recall from 0.00 to 0.18 without destabilising the other six classes, reaching a final weighted F1 of 0.60. Six concrete engineering pitfalls, from callback patience conflicts to a silently dropped class, are documented alongside their fixes as a practical reference. The trained model is integrated into a working real-time OpenCV pipeline with a verified, single-source-of-truth normalisation contract between training and inference. Future work could extend this baseline with a learned, rather than Haar cascade, face detector and a systematic real-time performance benchmark.

References

1. I. Goodfellow et al., "Challenges in Representation Learning: A Report on Three Machine Learning Contests" (FER-2013 dataset), 2013.
2. P. Viola and M. Jones, "Rapid Object Detection Using a Boosted Cascade of Simple Features," Proc. IEEE CVPR, 2001.
3. P. Urgunde, D. Gille, G. Hushangabadkar, N. Bokad, M. Shrikhande, and P. Kaware, "Real-Time Human Facial Emotion Recognition System Using Deep Learning," International Journal of Computer Techniques, vol. 13, no. 2, pp. 314–319, Mar.–Apr. 2026.
4. M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," Proc. USENIX OSDI, 2016.
5. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

6. G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.